NeoAlzette V6

A Long-Form, Implementation-Aligned Specification of a 64-bit ARX-box

Twilight-Dream[1]

Affiliation, City, Country, yujiang1187791459@gmail.com

Abstract. This document gives a long-form, implementation-aligned description of the NeoAlzette V6 64-bit ARX-box. The primitive operates on two 32-bit words and combines four structural ingredients: modular-addition-based ARX mixing, fixed cross-branch XOR-rotation coupling, input-dependent diffusion masks generated by lightweight linear maps, and asymmetric cross-branch injection gadgets built from boolean logic and rotated linear pre-mixing. The goal of this document is clarity rather than hype: it explains what the implementation actually does, what makes the construction distinct from a minimal textbook ARX box, and which properties are directly supported by the current source code. In particular, this document states the exact forward and backward transformations, the round constants, the two diffusion-mask generators, the two branch-injection procedures, and the implementation-level rationale for the chosen cross-rotation bridge. It also explicitly separates specification facts from design intent and does not make full security claims.

Keywords: ARX-box, symmetric cryptography, 64-bit primitive, implementation-aligned specification, diffusion layer, cross-branch injection

1 Introduction

NeoAlzette V6 is a 64-bit ARX-box working on a pair of 32-bit words (A, B). At first glance it belongs to the ARX family because it uses modular addition, rotation, and XOR as its main algebraic tools. However, the implemented construction is not just "a small ARX box with constants." Its current V6 form combines a short two-subround skeleton with an additional cross-branch injection mechanism. That injection mechanism is not a lookup-table S-box and not an external linear layer wrapped around the whole primitive. Instead, it is a local branch-to-branch mixing gadget derived from the current branch value itself.

The practical reason for writing this document in a long form is simple: a short specification can be correct yet still leave the reader with no intuition for what is special about the primitive. The current implementation contains several ideas that matter structurally:

- the box is only two subrounds long, but each subround contains more than a plain ARX update;
- the two branch-injection gadgets are asymmetric, rather than literally mirrored copies of one another;
- the "dynamic" masks are not runtime-generated random structures but deterministic input-dependent outputs of fixed linear maps;

- the V6 implementation deliberately removed older auxiliary linear layers and re-used the dynamic masks as the linear pre-mix inside the injector;
- the cross-rotation coupling constants were chosen with an explicit structural constraint on their sum modulo 32.

This document therefore serves three jobs at once:

1. to define the primitive unambiguously;
2. to explain the construction in ordinary technical language that can be read without reverse-engineering the C++ source;
3. to distinguish between facts that are directly implemented and broader claims that would still require separate cryptanalytic justification.

2 What This Document Claims and What It Does Not Claim

This is an implementation-aligned specification and design-description document. It *does* claim the following:

- the formulas match the current V6 core implementation;
- the primitive is reversible, because both forward and backward procedures are explicitly implemented;
- the two mask generators are fixed 32-bit linear maps over $\mathbb{F}_2$ built only from XOR and rotations;
- the source comments and the current implementation support specific structural motivations, such as avoiding large fixed-point periodic subspaces in the cross-rotation bridge;
- the implementation records concrete witness values and branch-number search results for the mask maps.

It *does not* claim the following:

- that NeoAlzette V6 is proven secure as a stand-alone cipher component in every cryptanalytic model;
- that the injection gadgets behave as formal pseudorandom functions;
- that the mask branch numbers alone imply full-round security;
- that a short implementation-level rationale is a substitute for a full differential, linear, algebraic, rotational, invariant-subspace, or related-key analysis.

That boundary matters. The current design is not “random words glued together.” There is a real structure in the implementation. But it is also not honest to turn every design motive into a theorem. This document stays on the sane side of that line.

3 High-Level Structure

NeoAlzette V6 takes a 64-bit state split into two 32-bit words (A, B). All additions and subtractions are performed modulo 2^{32}. All bitwise operations are on 32-bit words.

A single invocation of the box has three stages:

1. a first subround centered on updating B and then injecting a function of B back into A;
2. a second subround centered on updating A and then injecting a function of A back into B;
3. a final constant XOR on both branches.

Each subround itself has four conceptual layers:

1. an ARX update driven by rotated versions of the opposite branch and a constant;
2. a branch-local constant subtraction;
3. a fixed cross-branch XOR-rotation bridge;
4. a cross-branch injection gadget that computes two derived words (c, d) from one branch and injects rotated versions of them into the other branch.

So the box is compact in round count, but not minimal in structure. That is one of its defining characteristics.

4 Mathematical Setting and Primitive Operations

NeoAlzette V6 works over the word space $\mathbb{W}_{32} = \mathbb{Z}/2^{32}\mathbb{Z}$. The full state space is therefore

$$\mathbb{W}_{32}^2 = (\mathbb{Z}/2^{32}\mathbb{Z}) \times (\mathbb{Z}/2^{32}\mathbb{Z}),$$

and a state is written as a pair (A, B) with $A, B \in \mathbb{W}_{32}$.

The design belongs to the ARX/SRX family in the following concrete sense:

- A stands for modular addition over $\mathbb{Z}/2^{32}\mathbb{Z}$;
- S stands for modular subtraction over $\mathbb{Z}/2^{32}\mathbb{Z}$;
- R stands for cyclic bit rotation on 32-bit words;
- X stands for bitwise XOR on 32-bit words.

Thus, in this document:

- an *ARX expression* means an expression composed from modular addition, rotations, and XOR;
- an *SRX expression* means an expression composed from modular subtraction, rotations, and XOR;
- NeoAlzette V6 is most accurately described as an ARX/SRX-style word-oriented reversible primitive, because both addition and subtraction appear explicitly in the implemented core.

4.1 Word arithmetic

For $x, y \in \mathbb{W}_{32}$, define

$$x + y := (x + y) \bmod 2^{32},$$

$$x - y := (x - y) \bmod 2^{32}.$$

Equivalently,

$$x - y = x + (2^{32} - y) \pmod{2^{32}}.$$

So subtraction is not a separate universe: it is the modular inverse-form of addition on the same 32-bit word ring. It is still useful to write both explicitly in the specification, because the implementation uses both directions directly.

4.2 Bitwise operations

For $x, y \in \mathbb{W}_{32}$, the operations

$$x \oplus y, \qquad x \wedge y, \qquad x \vee y, \qquad \neg x$$

are the standard bitwise XOR, AND, OR, and NOT on the 32 individual bits of a 32-bit word.

If one writes

$$x = \sum_{i=0}^{31} x_i 2^i, \qquad y = \sum_{i=0}^{31} y_i 2^i,$$

with $x_i, y_i \in \{0, 1\}$, then

$$x \oplus y = \sum_{i=0}^{31} (x_i \oplus y_i) 2^i,$$

$$x \wedge y = \sum_{i=0}^{31} (x_i \wedge y_i) 2^i,$$

$$x \vee y = \sum_{i=0}^{31} (x_i \vee y_i) 2^i,$$

$$\neg x = \sum_{i=0}^{31} (1 - x_i) 2^i.$$

4.3 Rotations

For a 32-bit word x and an integer r, define the left and right cyclic rotations by

$$\mathrm{rotl}(x, r) = (x \ll r) \vee (x \gg (32 - r)),$$

$$\mathrm{rotr}(x, r) = (x \gg r) \vee (x \ll (32 - r)),$$

where r is interpreted modulo 32.

Equivalently, on bit indices these are the permutations

$$(\mathrm{rotl}(x, r))_i = x_{(i-r) \bmod 32},$$

$$(\mathrm{rotr}(x, r))_i = x_{(i+r) \bmod 32}.$$

4.4 ARX/SRX viewpoint for this specification

The reason these definitions are stated explicitly is that NeoAlzette V6 mixes two different kinds of nonlinearity and one main linear transport mechanism:

- modular addition and subtraction contribute carry/borrow-driven nonlinearity over $\mathbb{Z}/2^{32}\mathbb{Z}$;
- XOR and rotations provide word-level linear transport over $\mathbb{F}_2^{32}$;
- the branch-injection gadgets add extra local boolean nonlinearity through AND, OR, and NOT.

So the box is not merely "ARX in the narrowest slogan sense." At the implementation level it is a reversible ARX/SRX word construction with additional boolean branch injection.

5 Round Constants

Let RC[0..15] be the following 16-element array of 32-bit constants:

```
RC = [
 0x16B2C40B, 0xC117176A, 0x0F9A2598, 0xA1563ACA,
 0x243F6A88, 0x85A308D3, 0x13198102, 0xE0370734,
 0x9E3779B9, 0x7F4A7C15, 0xF39CC060, 0x5CEDC834,
 0xB7E15162, 0x8AED2A6A, 0xBF715880, 0x9CF4F3C7
]
```

The implementation comments indicate that these constants are drawn from several recognizable numeric sources: a concatenated Fibonacci-derived prefix, followed by words associated with the hexadecimal expansions of π, φ, and e.

In the current box core, only RC[0] through RC[11] are actively used. The remaining constants are currently reserved.

6 Cross-Branch XOR-Rotation Bridge

Between the add/sub step and the injection step, each subround applies a fixed two-line cross-coupling:

$$A \leftarrow A \oplus \mathrm{rotl}(B, R_0^{cross}),$$
$$B \leftarrow B \oplus \mathrm{rotl}(A, R_1^{cross}),$$

or the analogous branch-reversed version in the second subround.

In the current V6 instantiation, the ordered bridge constants are fixed as

$$R_0^{cross} = 22, \qquad R_1^{cross} = 13.$$

Equivalently, the underlying screened rotation pair is $(13, 22)$, with the implementation using the ordered assignment $(R_0, R_1) = (22, 13)$. Therefore,

$$R_0^{cross} + R_1^{cross} \equiv 22 + 13 \equiv 35 \equiv 3 \pmod{32}.$$

These constants were not fixed as arbitrary decoration. In an earlier stage, we experimented with more loosely chosen rotation pairs, but large-scale automated ARX cryptanalysis on 32-bit words repeatedly exposed undesirable structural behavior for several such choices. In particular, some rotation settings created alignment effects that

made the injection layer easier to bypass, after which the best trails often retained only a very small number of active modular additions. Since modular addition is the main nonlinear source in the ARX setting, this phenomenon was treated as a structural warning sign rather than a benign implementation artifact. The final screening rule was therefore introduced as a corrective design step.

More precisely, let the candidate pair be (r_1, r_2) over 32-bit words. We require

$$\gcd(((r_1 + r_2) \bmod 32), 32) = 1.$$

Since the modulus is $32 = 2^5$, this is equivalent to requiring $(r_1 + r_2) \bmod 32$ to be odd. This condition is also reflected in the implementation by an explicit static assertion enforcing that the bridge-sum modulo 32 must be odd. The structural motivation is that if the sum had a large common divisor with 32, then there would exist large periodic rotation subspaces in which expressions of the form

$$dB \oplus \mathrm{rotl}(dB, R_0^{cross} + R_1^{cross})$$

could vanish too easily, allowing parts of the bridge to become transparent under single-branch XOR-difference propagation. In plain language, the bridge is screened to avoid obvious large fixed-pattern escape hatches.

In addition, we explicitly exclude the values 8, 16, and 24. These correspond to the quarter-turn, half-turn, and three-quarter-turn positions of a 32-bit word, and repeatedly exhibited undesirable structural behavior in our automated searches. Candidates involving these positions were more likely to produce bypass-like alignments, leaving too few modular additions active in the resulting best trails.

Among the remaining candidates, we further rank pairs by their circular separation

$$d(r_1, r_2) = \min\{(r_2 - r_1) \bmod 32,\ (r_1 - r_2) \bmod 32\},$$

and prefer candidates satisfying

$$5 \le d(r_1, r_2) \le 11.$$

This heuristic avoids both overly local rotation gaps and near half-turn symmetry. We also prefer pairs spanning different halves of the word, i.e., one rotation in $[1, 15]$ and the other in $[17, 31]$, in order to reduce same-region clustering and discourage visually symmetric or structurally fragile layouts. These rules are not presented as a security proof; rather, they are experimentally grounded design filters intended to remove rotation configurations that consistently appeared weak under automated analysis.

The final screened choice is the pair

$$(r_1, r_2) = (13, 22),$$

which yields the implemented ordered constants $(R_0, R_1) = (22, 13)$. This choice avoids the excluded positions $\{8, 16, 24\}$, spans different halves of the 32-bit word, has circular distance

$$d(13, 22) = 9,$$

and satisfies

$$(13 + 22) \bmod 32 = 3,$$

which is coprime to 32. Empirically, this gives a balanced and non-degenerate bridge configuration under our screening methodology, while avoiding the experimentally fragile quarter-turn and half-turn related positions observed in the automated search results.

That does *not* prove that no useful differential or linear structures exist. It only means that the implemented parameter choice is backed by a concrete structural concern and by automated cryptanalytic evidence, rather than by blind numerology.

7 Input-Dependent Diffusion Masks

A central feature of V6 is the use of two deterministic input-dependent diffusion masks, one built from left rotations and one built from right rotations.

7.1 Mask mask0

The first mask generator is:

```
v0 = X
v1 = v0 ^ rotl(v0,2)
v2 = v0 ^ rotl(v1,17)
v3 = v0 ^ rotl(v2,4)
v4 = v3 ^ rotl(v3,24)
mask0(X) = v2 ^ rotl(v4,7)
```

Equivalently,

$$\mathsf{mask0}(X) = v_2 \oplus \mathrm{rotl}(v_4, 7),$$

with the intermediate variables defined as above.

7.2 Mask mask1

The second mask generator is:

```
v0 = X
v1 = v0 ^ rotr(v0,2)
v2 = v0 ^ rotr(v1,17)
v3 = v0 ^ rotr(v2,4)
v4 = v3 ^ rotr(v3,24)
mask1(X) = v2 ^ rotr(v4,7)
```

Equivalently,

$$\mathsf{mask1}(X) = v_2 \oplus \mathrm{rotr}(v_4, 7).$$

7.3 What "dynamic" means here

The term "dynamic" can sound more mysterious than it is. In this implementation it does *not* mean the linear map itself changes during execution. The structure is fixed. What changes is the output because the input word changes. So these masks are better understood as *deterministic input-dependent linear transforms*.

7.4 Linear-map interpretation

Because these functions use only XOR and rotations, both are linear over $\mathbb{F}_2^{32}$. Hence there exist binary 32×32 matrices M_0 and M_1 such that

$$\mathsf{mask0}(x) = M_0 x, \qquad \mathsf{mask1}(x) = M_1 x.$$

The implementation comments record the image of the basis vector $e_0 =$ `0x00000001`:

$$M_0 e_0 = \texttt{0xd05a0889}, \qquad M_1 e_0 = \texttt{0x2220b417}.$$

Because the maps are built from rotations and XOR only, they are rotation-equivariant and can be viewed as circulant linear maps generated from those basis images.

7.5 Recorded branch-number evidence

The implementation comments also record a search result for these two mask constructions. The reported exact bit-level differential and linear branch numbers are both 12 for each map, with witness input `0x00000001`.

Formally, for a linear map M on 32-bit words, define the bit-level branch metric

$$B(M) = \min_{x \neq 0} \big(\mathrm{wt}(x) + \mathrm{wt}(Mx)\big),$$

where $\mathrm{wt}(\cdot)$ is Hamming weight.

The current implementation comments report:

$$B_\Delta(M_0) = 12, \qquad B_\lambda(M_0) = 12,$$

$$B_\Delta(M_1) = 12, \qquad B_\lambda(M_1) = 12.$$

This is an implementation-supported statement about the mask layers themselves. It is meaningful, because it shows the chosen 5-XOR-style linear maps are not flimsy. But it should still be read as a property of those linear components, not a complete evaluation of the whole box.

8 Cross-Branch Injection Gadgets

The most distinctive part of NeoAlzette V6 is the pair of cross-branch injection gadgets. In the current implementation these gadgets are not treated as separate reversible mini-permutations. Instead, they are lightweight branch-derived boolean mixers whose outputs are xored into the opposite branch inside a reversible ARX/SRX round.

Their implementation-level design role is:

- add a second local nonlinearity source beyond the carry/borrow effects of modular addition and subtraction;
- remain lightweight in terms of word operations and precomputed constants;
- inject branch-derived structure across branches without inserting a heavyweight external substitution layer.

8.1 Precomputed constants used by the current implementation

The present implementation precomputes

$$\mathrm{RC7_R24} = \mathrm{rotr}(\mathrm{RC}[7], 24), \quad \mathrm{RC8_R24} = \mathrm{rotr}(\mathrm{RC}[8], 24), \quad \mathrm{RC13_R24} = \mathrm{rotr}(\mathrm{RC}[13], 24),$$

$$\mathrm{RC2_L8} = \mathrm{rotl}(\mathrm{RC}[2], 8), \quad \mathrm{RC3_L8} = \mathrm{rotl}(\mathrm{RC}[3], 8), \quad \mathrm{RC12_L8} = \mathrm{rotl}(\mathrm{RC}[12], 8),$$

and the constant diffusion-mask images

$$\mathrm{MASK0_RC7} = \mathsf{mask0}(\mathrm{RC}[7]), \qquad \mathrm{MASK1_RC2} = \mathsf{mask1}(\mathrm{RC}[2]).$$

It also uses the bridge helper

$$\mathsf{bridge}(x) = x \oplus \mathrm{rotl}(\mathrm{rotl}(x, 22), 13).$$

8.2 Current implementation of the $B \to A$ injector

The first injector operates on the already-localized source word B after the subround has applied

$$B \leftarrow B \oplus \mathrm{RC}[4].$$

It then defines

$$B_2 = \mathrm{rotr}(B, 24), \qquad M_B = \mathsf{mask0}(B), \qquad M_B^{(5)} = \mathrm{rotr}(M_B, 5),$$

$$\overline{M}_B = \mathrm{rotr}(M_B, 24) \oplus \mathrm{MASK0_RC7}, \qquad x_{0,B} = B_2 \oplus M_B, \qquad x_{1,B} = B \oplus M_B,$$

$$v_B = B_2 \oplus \overline{M}_B, \qquad g_B = \mathsf{bridge}(B).$$

From these it computes

$$q_{B,\mathrm{state}} = \mathrm{RC7_R24} \oplus \neg(B \wedge M_B),$$

$$q_{B,\mathrm{comp}} = B_2 \oplus B \oplus \mathrm{RC8_R24} \oplus \neg(B_2 \vee M_B^{(5)}),$$

$$q_{B,\mathrm{bridge}} = g_B \oplus B \oplus \mathrm{RC13_R24} \oplus \neg(g_B \wedge \overline{M}_B),$$

and the shared carrier

$$q_{B,\mathrm{shared}} = q_{B,\mathrm{state}} \oplus q_{B,\mathrm{comp}}.$$

The side terms are

$$\mathit{cross}_B = (B \oplus M_B^{(5)}) \wedge \mathrm{rotr}(M_B \oplus \overline{M}_B, 7),$$

$$\mathit{anti}_B = \big((x_{1,B} \gg 3) \oplus (v_B \gg 5) \oplus M_B^{(5)}\big) \wedge \big(q_{B,\mathrm{comp}} \oplus \mathrm{rotr}(x_{0,B}, 11)\big).$$

The returned pair is

$$C_B = q_{B,\mathrm{shared}} \oplus \mathrm{rotr}(q_{B,\mathrm{comp}}, 5) \oplus \mathrm{rotr}(q_{B,\mathrm{comp}}, 11) \oplus \mathit{anti}_B,$$

$$D_B = q_{B,\mathrm{shared}} \oplus \mathrm{rotr}(q_{B,\mathrm{state}}, 5) \oplus \mathrm{rotr}(q_{B,\mathrm{bridge}}, 13) \oplus \mathit{cross}_B \oplus \mathit{anti}_B.$$

The value injected into the opposite branch is

$$f_B(B) = \mathrm{rotl}\big((C_B \ll 2) \oplus (D_B \gg 2), 24\big) \oplus \mathrm{rotl}(D_B, 16) \oplus \mathrm{rotl}\big((C_B \gg 5) \oplus (D_B \ll 5), 8\big).$$

8.3 Current implementation of the $A \to B$ injector

The second injector operates on the already-localized source word A after the subround has applied

$$A \leftarrow A \oplus \mathrm{RC}[9].$$

It defines

$$A_2 = \mathrm{rotl}(A, 8), \qquad M_A = \mathsf{mask1}(A), \qquad M_A^{(5)} = \mathrm{rotr}(M_A, 5),$$

$$\overline{M}_A = \mathrm{rotl}(M_A, 8) \oplus \mathrm{MASK1_RC2}, \qquad x_{0,A} = A_2 \oplus M_A, \qquad x_{1,A} = A \oplus M_A,$$

$$v_A = A_2 \oplus \overline{M}_A, \qquad g_A = \mathsf{bridge}(A).$$

Then

$$q_{A,\mathrm{state}} = \mathrm{RC2_L8} \oplus \neg(A \vee M_A),$$

$$q_{A,\mathrm{comp}} = A_2 \oplus A \oplus \mathrm{RC3_L8} \oplus \neg(A_2 \wedge M_A^{(5)}),$$

$$q_{A,\mathrm{bridge}} = g_A \oplus A \oplus \mathrm{RC12_L8} \oplus \neg(g_A \vee \overline{M}_A),$$

with shared carrier

$$q_{A,\text{shared}} = q_{A,\text{state}} \oplus q_{A,\text{comp}}.$$

The side terms are

$$cross_A = (A \oplus M_A^{(5)}) \wedge \text{rotl}(M_A \oplus \overline{M}_A, 13),$$

$$anti_A = \big((x_{1,A} \ll 3) \oplus (v_A \ll 5) \oplus M_A^{(5)}\big) \vee \big(q_{A,\text{comp}} \oplus \text{rotl}(x_{0,A}, 11)\big).$$

The returned pair is

$$C_A = q_{A,\text{shared}} \oplus \text{rotl}(q_{A,\text{comp}}, 5) \oplus \text{rotl}(q_{A,\text{comp}}, 11) \oplus anti_A,$$

$$D_A = q_{A,\text{shared}} \oplus \text{rotl}(q_{A,\text{state}}, 5) \oplus \text{rotl}(q_{A,\text{bridge}}, 13) \oplus cross_A \oplus anti_A.$$

The injected value is

$$f_A(A) = \text{rotr}\big((C_A \gg 3) \oplus (D_A \ll 3), 24\big) \oplus \text{rotr}(D_A, 16) \oplus \text{rotr}\big((C_A \ll 1) \oplus (D_A \gg 1), 8\big).$$

8.4 Why the two directions are intentionally asymmetric

The two injectors share a common design philosophy, but they are not literal mirrors:

- the companion views differ: $\text{rotr}(\cdot, 24)$ on the $B \to A$ side and $\text{rotl}(\cdot, 8)$ on the $A \to B$ side;
- the constant-mask anchors differ: MASK0_RC7 versus MASK1_RC2;
- one side is organized around complemented AND-form state terms and complemented OR-form companion terms, while the other swaps these roles;
- the shift directions and outer projection formulas are intentionally different.

8.5 Recorded low-weight rank property

This document is a specification, not the formal analysis paper. Therefore it does *not* reproduce the exact matrix-construction algorithms, witness enumeration procedures, or low-weight search methodology. For specification purposes it is enough to record the implementation-facing conclusion:

- standalone linear minimum rank: 28;
- contextual linear minimum rank: 28;
- standalone differential minimum rank: 28;
- contextual differential minimum rank: 28;
- no structured zero-output witness recorded in the validated low-Hamming-weight window.

The exact rank derivation and witness logs belong to the separate formal analysis paper rather than to this implementation-oriented specification.

9 Why NeoAlzette V6 is not pure ARX/SRX, and why that is intentional

A natural question is whether the injection layer is superfluous, or whether its use moves the construction too far away from the usual ARX/SRX design philosophy.

The short answer is: NeoAlzette V6 is not a *strict* pure-ARX or pure-SRX box in the narrow acronym sense, because its branch injectors explicitly use small boolean gates beyond addition, subtraction, rotation, and XOR. However, this does not make the construction philosophically alien to ARX/SRX-style design. The implemented primitive remains table-free, word-oriented, round-reversible, and built entirely from lightweight machine-friendly operations. The most accurate description is therefore not "textbook pure ARX", but rather an *ARX/SRX-core construction with a lightweight auxiliary boolean injection layer*.

This boolean design choice is deliberate. Its inspiration comes from the style of lightweight combinational S-boxes used in designs such as ASCON, where local nonlinearity is realized directly by compact boolean formulas rather than by a lookup table. In that sense, the injector is S-box-like, but it is not a classical table-based S-box. Instead, it is a 32-bit branch-local, mask-coupled, table-free boolean mixing gadget embedded into a larger reversible ARX/SRX round.

What matters algebraically is not the implementation variable name `s_box_in_*`, but the actual degree structure. The complemented terms only modify affine carriers, while the degree-2 kernel is created at explicit product sites between linear carriers. So the point of combining AND-form logic, OR-form logic, and complemented terms is not to imitate a textbook substitution layer literally. Rather, it is to import the lightweight boolean-combinational design philosophy of ASCON-style local layers into a word-oriented cross-branch injector, while also preventing the two directional injectors from degenerating into a trivial mirrored copy.

At the whole-round level, the main ARX/SRX nonlinearity still comes from modular addition and subtraction through carry/borrow propagation. The branch injector is added as a second local boolean source beyond that main path, not as a replacement for it. Its role is therefore auxiliary rather than dominant: it enriches the local algebraic structure without turning the primitive into a table-driven design.

From a performance viewpoint, the layer is visibly richer than a minimal pure-ARX core, but it is still lightweight in implementation terms. It introduces no lookup tables, no finite-field multipliers, and no large external substitution layer. Its cost is paid entirely in short 32-bit word operations together with two small fixed diffusion-mask circuits. So while the construction is not pure ARX in the narrow acronym sense, it still respects the broader implementation philosophy that makes ARX/SRX-style primitives attractive in practice.

Accordingly, the right conclusion is not that the injection layer is a flaw. The right conclusion is that NeoAlzette V6 deliberately departs from strict acronym purity in order to import a lightweight ASCON-style boolean-combinational local layer into an otherwise ARX/SRX-core word construction.

This is not an attempt to claim that the original Alzette was lacking something essential. On the contrary, the original Alzette algorithm is already exceptionally strong and efficient: it uses only four modular additions/subtractions; 1 round can already achieve Rijndael S-box strength, and 2 rounds can achieve super-Rijndael-S-box strength. Precisely because that baseline is already so good, any departure from it must be justified as a deliberate tradeoff rather than as accidental complication.

The auxiliary boolean injection layer should therefore be understood in exactly that way. It is not introduced because the original Alzette is inadequate, and it is not meant to imitate a classical substitution layer at full cost. Rather, once one decides not to

remain identical to the original design, some performance price must be paid in exchange for additional structural mixing. Among realistic lightweight choices, a small table-free boolean-combinational layer is one of the few ways to add an extra local quadratic source without moving all the way to a heavyweight external S-box architecture.

In that sense, the injection layer occupies an intermediate design point: it is clearly richer than a minimal pure-ARX core, but still much lighter than attaching a heavyweight external substitution layer.

So the intended innovation point is not "Alzette was missing something and we repaired it". The intended point is that NeoAlzette V6 explores a different design point: it keeps an ARX/SRX-style word-oriented core, accepts a controlled performance overhead, and uses that budget to import an additional ASCON-inspired local boolean mixing layer. In that sense, the injector is neither a negligible decoration nor a heavyweight external S-box replacement; it is an intermediate table-free local layer chosen as a conscious structural tradeoff.

10 Relationship to Older Internal Drafts

The V6 implementation explicitly removed older auxiliary linear layers commonly referred to as L_1 and L_2 in earlier internal drafts.

That removal is not a vague rewrite. The implementation comments state the intended replacement clearly: instead of paying for extra outer linear layers around the injector, V6 reuses the already-computed dynamic diffusion masks as the injector's linear pre-mix.

In practical terms, this means:

- fewer extra rotate/XOR operations than a design that wraps the injector in separate standalone linear layers;
- a tighter specification, because the linear pre-mix is no longer conceptually detached from the injection gadget;
- a more coherent V6 identity, where the same mask-generating substructure serves both diffusion and branch-derived injection preparation.

So if someone says "V6 is just noise," that would be inaccurate. There is a visible design simplification here: older extra layers were removed, and the remaining structure was consolidated rather than arbitrarily bloated.

11 Forward Transformation

Given input words (A, B), the forward direction of one NeoAlzette V6 box is defined as follows.

11.1 First subround

1. ARX update on B.

$$B \leftarrow B + \big(\mathrm{rotl}(A, 31) \oplus \mathrm{rotl}(A, 17) \oplus \mathrm{RC}[0]\big).$$

2. Constant subtraction on A.

$$A \leftarrow A - \mathrm{RC}[1].$$

3. Cross-branch XOR-rotation bridge.

$$A \leftarrow A \oplus \mathrm{rotl}(B, R_0^{cross}),$$

$$B \leftarrow B \oplus \mathrm{rotl}(A, R_1^{cross}).$$

4. Injection from B into A. Compute

$$(C_0, D_0) = \mathrm{cd_injection_from_B}\big(B, (\mathrm{RC}[2] \vee \mathrm{RC}[3]), \mathrm{RC}[3]\big).$$

Then update

$$A \leftarrow A \oplus \mathrm{rotl}(C_0, 24) \oplus \mathrm{rotl}(D_0, 16) \oplus \mathrm{rotl}\Big(\big(C_0 \gg 1\big) \oplus \big(D_0 \ll 1\big), 8\Big) \oplus \mathrm{RC}[4].$$

11.2 Second subround

1. ARX update on A.

$$A \leftarrow A + \big(\mathrm{rotl}(B, 31) \oplus \mathrm{rotl}(B, 17) \oplus \mathrm{RC}[5]\big).$$

2. Constant subtraction on B.

$$B \leftarrow B - \mathrm{RC}[6].$$

3. Cross-branch XOR-rotation bridge.

$$B \leftarrow B \oplus \mathrm{rotl}(A, R_0^{cross}),$$

$$A \leftarrow A \oplus \mathrm{rotl}(B, R_1^{cross}).$$

4. Injection from A into B. Compute

$$(C_1, D_1) = \mathrm{cd_injection_from_A}\big(A, (\mathrm{RC}[7] \wedge \mathrm{RC}[8]), \mathrm{RC}[8]\big).$$

Then update

$$B \leftarrow B \oplus \mathrm{rotr}(C_1, 24) \oplus \mathrm{rotr}(D_1, 16) \oplus \mathrm{rotr}\Big(\big(C_1 \ll 3\big) \oplus \big(D_1 \gg 3\big), 8\Big) \oplus \mathrm{RC}[9].$$

11.3 Final whitening-style XOR

Finally,

$$A \leftarrow A \oplus \mathrm{RC}[10], \qquad B \leftarrow B \oplus \mathrm{RC}[11].$$

The resulting pair (A, B) is the output of the box.

12 Backward Transformation

The implementation also provides the explicit reverse transformation. This matters because reversibility is not hand-waved; it is concretely defined by code.

Given output words (A, B), the backward direction is:

12.1 Undo final XOR

$$B \leftarrow B \oplus \mathrm{RC}[11], \qquad A \leftarrow A \oplus \mathrm{RC}[10].$$

12.2 Reverse the second subround

First recompute the injector output from the current A:

$$(C_1, D_1) = \text{cd_injection_from_A}\big(A, (\text{RC}[7] \wedge \text{RC}[8]), \text{RC}[8]\big).$$

Then undo the injection contribution:

$$B \leftarrow B \oplus \text{rotr}(C_1, 24) \oplus \text{rotr}(D_1, 16) \oplus \text{rotr}\Big(\big(C_1 \ll 3\big) \oplus \big(D_1 \gg 3\big), 8\Big) \oplus \text{RC}[9].$$

Undo the bridge:

$$A \leftarrow A \oplus \text{rotl}(B, R_1^{cross}),$$
$$B \leftarrow B \oplus \text{rotl}(A, R_0^{cross}).$$

Undo the subtraction and ARX update:

$$B \leftarrow B + \text{RC}[6],$$

$$A \leftarrow A - \big(\text{rotl}(B, 31) \oplus \text{rotl}(B, 17) \oplus \text{RC}[5]\big).$$

12.3 Reverse the first subround

Recompute the injector output from the current B:

$$(C_0, D_0) = \text{cd_injection_from_B}\big(B, (\text{RC}[2] \vee \text{RC}[3]), \text{RC}[3]\big).$$

Undo the injection contribution:

$$A \leftarrow A \oplus \text{rotl}(C_0, 24) \oplus \text{rotl}(D_0, 16) \oplus \text{rotl}\Big(\big(C_0 \gg 1\big) \oplus \big(D_0 \ll 1\big), 8\Big) \oplus \text{RC}[4].$$

Undo the bridge:

$$B \leftarrow B \oplus \text{rotl}(A, R_1^{cross}),$$
$$A \leftarrow A \oplus \text{rotl}(B, R_0^{cross}).$$

Undo the subtraction and ARX update:

$$A \leftarrow A + \text{RC}[1],$$

$$B \leftarrow B - \big(\text{rotl}(A, 31) \oplus \text{rotl}(A, 17) \oplus \text{RC}[0]\big).$$

At that point the original input pair is recovered.

13 What Makes NeoAlzette V6 Distinct

This section states, in plain technical language, what is visibly distinctive about the current implementation.

13.1 It is not just a plain short ARX box

A minimal ARX box would usually stop after a few addition/XOR/rotation lines and constants. NeoAlzette V6 instead inserts a branch-derived injection gadget in each subround. That means local boolean mixing is not coming only from the carry and borrow chains of modular arithmetic.

13.2 Its extra structure is still lightweight

The added machinery is built from XOR, rotations, AND, OR, NOT, and a few constants. There is no table lookup, no large matrix multiplication written as a dense external object, and no heavyweight nonlinear component in the usual block-cipher sense.

13.3 The diffusion masks are structured, not ad hoc

The masks are not random one-off formulas tossed into the code. They are fixed linear maps built from a short XOR-rotation circuit, and the implementation comments preserve concrete search evidence for their branch properties.

13.4 The cross-branch bridge was constrained for a reason

The pair $(13, 22)$ is backed by an explicit implementation-level criterion on $R_0 + R_1 \bmod 32$. So the bridge was not chosen as decorative numerology.

13.5 The two branch injectors are intentionally asymmetric

The design uses AND-form logic on one side and OR-form logic on the other, along with opposite rotation directions in the internal update. That is a real structural choice, not empty buzzword smoke.

13.6 V6 is a simplification, not a random complication

By removing earlier L_1/L_2 wrappers and reusing the mask circuits as the injector's linear pre-mix, V6 consolidates structure instead of endlessly stacking layers.

14 What One Can Honestly Say About the Design

Based on the current implementation, the following are reasonable and defensible statements.

- NeoAlzette V6 is a reversible 64-bit ARX-box with two asymmetric branch-injection stages.
- Its nonlinearity is not sourced only from modular carry/borrow behavior; the injector adds an additional local boolean mixing mechanism.
- Its dynamic masks are deterministic linear transforms with recorded branch-number evidence in the implementation comments.
- Its cross-branch bridge uses a parameter condition chosen to avoid an obvious class of large periodic fixed-pattern subspaces.
- Its V6 form reflects a concrete simplification step relative to older drafts that used extra outer linear layers.

Those are fair statements.

By contrast, the following would be too strong if presented as settled fact in a specification-only document:

- "NeoAlzette V6 is proven secure."
- "The injector guarantees resistance against all useful differential trails."

- "Branch number 12 for the mask maps implies full-box security."
- "The local boolean mix behaves like an ideal S-box."

So no, the current implementation is not "talking nonsense." But yes, the honest version is still narrower than a full security proof.

15 Compact Pseudocode Summary

Algorithm 1 Dynamic diffusion mask mask0

```
function MASK0(X)
    v_0 ← X
    v_1 ← v_0 ⊕ rotl(v_0, 2)
    v_2 ← v_0 ⊕ rotl(v_1, 17)
    v_3 ← v_0 ⊕ rotl(v_2, 4)
    v_4 ← v_3 ⊕ rotl(v_3, 24)
    return v_2 ⊕ rotl(v_4, 7)
end function
```

Algorithm 2 Dynamic diffusion mask mask1

```
function MASK1(X)
    v_0 ← X
    v_1 ← v_0 ⊕ rotr(v_0, 2)
    v_2 ← v_0 ⊕ rotr(v_1, 17)
    v_3 ← v_0 ⊕ rotr(v_2, 4)
    v_4 ← v_3 ⊕ rotr(v_3, 24)
    return v_2 ⊕ rotr(v_4, 7)
end function
```

Algorithm 3 Injection from B into A

```
function CDINJECTIONFROMB(B)
    B_2 ← rotr(B, 24)
    M_B ← MASK0(B)
    M_B^(5) ← rotr(M_B, 5)
    M̄_B ← rotr(M_B, 24) ⊕ MASK0_RC7
    x_{0,B} ← B_2 ⊕ M_B
    x_{1,B} ← B ⊕ M_B
    v_B ← B_2 ⊕ M̄_B
    g_B ← B ⊕ rotl(rotl(B, 22), 13)
    q_{B,state} ← RC7_R24 ⊕ ¬(B ∧ M_B)
    q_{B,comp} ← B_2 ⊕ B ⊕ RC8_R24 ⊕ ¬(B_2 ∨ M_B^(5))
    q_{B,bridge} ← g_B ⊕ B ⊕ RC13_R24 ⊕ ¬(g_B ∧ M̄_B)
    q_{B,shared} ← q_{B,state} ⊕ q_{B,comp}
    cross_B ← (B ⊕ M_B^(5)) ∧ rotr(M_B ⊕ M̄_B, 7)
    anti_B ← ((x_{1,B} ≫ 3) ⊕ (v_B ≫ 5) ⊕ M_B^(5)) ∧ (q_{B,comp} ⊕ rotr(x_{0,B}, 11))
    C_B ← q_{B,shared} ⊕ rotr(q_{B,comp}, 5) ⊕ rotr(q_{B,comp}, 11) ⊕ anti_B
    D_B ← q_{B,shared} ⊕ rotr(q_{B,state}, 5) ⊕ rotr(q_{B,bridge}, 13) ⊕ cross_B ⊕ anti_B
    return (C_B, D_B)
end function
```

Algorithm 4 Injection from A into B

```
function CdInjectionFromA(A)
    A_2 ← rotl(A, 8)
    M_A ← Mask1(A)
    M_A^(5) ← rotr(M_A, 5)
    M̄_A ← rotl(M_A, 8) ⊕ MASK1_RC2
    x_{0,A} ← A_2 ⊕ M_A
    x_{1,A} ← A ⊕ M_A
    v_A ← A_2 ⊕ M̄_A
    g_A ← A ⊕ rotl(rotl(A, 22), 13)
    q_{A,state} ← RC2_L8 ⊕ ¬(A ∨ M_A)
    q_{A,comp} ← A_2 ⊕ A ⊕ RC3_L8 ⊕ ¬(A_2 ∧ M_A^(5))
    q_{A,bridge} ← g_A ⊕ A ⊕ RC12_L8 ⊕ ¬(g_A ∨ M̄_A)
    q_{A,shared} ← q_{A,state} ⊕ q_{A,comp}
    cross_A ← (A ⊕ M_A^(5)) ∧ rotl(M_A ⊕ M̄_A, 13)
    anti_A ← ((x_{1,A} ≪ 3) ⊕ (v_A ≪ 5) ⊕ M_A^(5)) ∨ (q_{A,comp} ⊕ rotl(x_{0,A}, 11))
    C_A ← q_{A,shared} ⊕ rotl(q_{A,comp}, 5) ⊕ rotl(q_{A,comp}, 11) ⊕ anti_A
    D_A ← q_{A,shared} ⊕ rotl(q_{A,state}, 5) ⊕ rotl(q_{A,bridge}, 13) ⊕ cross_A ⊕ anti_A
    return (C_A, D_A)
end function
```

Algorithm 5 Forward transformation

```
function Forward(A, B)
    B ← B + (rotl(A, 31) ⊕ rotl(A, 17) ⊕ RC[0])
    A ← A − RC[1]
    A ← A ⊕ rotl(B, 22)
    B ← B ⊕ rotl(A, 13)
    B ← B ⊕ RC[4]
    (C_0, D_0) ← CdInjectionFromB(B)
    A ← A⊕rotl((C_0 ≪ 2)⊕(D_0 ≫ 2), 24)⊕rotl(D_0, 16)⊕rotl((C_0 ≫ 5)⊕(D_0 ≪ 5), 8)
    A ← A + (rotl(B, 31) ⊕ rotl(B, 17) ⊕ RC[5])
    B ← B − RC[6]
    B ← B ⊕ rotl(A, 22)
    A ← A ⊕ rotl(B, 13)
    A ← A ⊕ RC[9]
    (C_1, D_1) ← CdInjectionFromA(A)
    B ← B⊕rotr((C_1 ≫ 3)⊕(D_1 ≪ 3), 24)⊕rotr(D_1, 16)⊕rotr((C_1 ≪ 1)⊕(D_1 ≫ 1), 8)
    A ← A ⊕ RC[10]
    B ← B ⊕ RC[11]
    return (A, B)
end function
```

Algorithm 6 Backward transformation

```
function Backward(A, B)
    B ← B ⊕ RC[11]
    A ← A ⊕ RC[10]
    A ← A ⊕ RC[9]
    (C_1, D_1) ← CdInjectionFromA(A)
    B ← B⊕rotr((C_1 ≫ 3)⊕(D_1 ≪ 3), 24)⊕rotr(D_1, 16)⊕rotr((C_1 ≪ 1)⊕(D_1 ≫ 1), 8)
    A ← A ⊕ rotl(B, 13)
    B ← B ⊕ rotl(A, 22)
    B ← B + RC[6]
```

```
    A ← A − (rotl(B, 31) ⊕ rotl(B, 17) ⊕ RC[5])
    B ← B ⊕ RC[4]
    (C_0, D_0) ← CdInjectionFromB(B)
    A ← A ⊕ rotl((C_0 ≪ 2) ⊕ (D_0 ≫ 2), 24) ⊕ rotl(D_0, 16) ⊕ rotl((C_0 ≫ 5) ⊕ (D_0 ≪ 5), 8)
    B ← B ⊕ rotl(A, 13)
    A ← A ⊕ rotl(B, 22)
    A ← A + RC[1]
    B ← B − (rotl(A, 31) ⊕ rotl(A, 17) ⊕ RC[0])
    return (A, B)
end function
```

16 Conclusion

NeoAlzette V6 is best understood as a compact ARX-box that deliberately goes one step beyond a bare ARX skeleton. Its current implementation combines ARX updates, a structurally constrained cross-rotation bridge, two deterministic linear diffusion-mask generators, and asymmetric branch-derived boolean injectors. The construction therefore has a visible internal logic: it is not merely a bag of unrelated tricks, and the V6 revision is not a random pile-up of layers. At the same time, the right way to present it is with disciplined wording. One can confidently describe its implemented structure, its reversible forward/backward behavior, the exact role of its mask generators, the recorded branch-number evidence for those linear masks, and the design intent behind the chosen bridge and injector arrangement. One should not inflate those facts into complete security claims without further cryptanalytic work.

That is the clean line. On the implementation side, NeoAlzette V6 has real structure and real distinguishing features. On the proof side, it still deserves the same hard analysis as any other serious ARX-derived primitive.

Notes

- This document is aligned to the current V6 core implementation.
- The text is intentionally long-form and explanatory, so that a reader does not need to infer the design from raw C++ alone.
- No benchmark section or test-vector appendix is included here.
- No claim is made here beyond what is directly supported by the current implementation and its recorded comments.